Appendix A: Tooling and Reproduction

Appendix A: Tooling and Reproduction I

The pipeline is a two-layer system: generic infrastructure (rendering, validation, logging) shared across the template monorepo, and project-local `src/` modules that implement the multi-phase literature review. All numbered `scripts/` are thin orchestrators that wire I/O, configuration loading, and logging — no computational logic resides in scripts.

Reproduce the Offline Default Run I

No network, no language model required. The default analysis lane replays the tracked three-phase evidence snapshot, then runs deterministic analysis, figure generation, assessment, evaluation, deep-research replay, bibliography export, and variable injection:

```
uv run python projects/templates/template_advanced_li
terature_review/scripts/01b_fixture_phase_replay.py
uv run python projects/templates/template_advanced_li
terature_review/scripts/02_meta_analysis_pipeline.py
uv run python projects/templates/template_advanced_li
terature_review/scripts/04_generate_figures.py --dpi
300
uv run python projects/templates/template_advanced_li
terature_review/scripts/06_fulltext_assessment.py
uv run python projects/templates/template_advanced_li
terature_review/scripts/07_literature_evaluation.py
uv run python projects/templates/template_advanced_li
terature_review/scripts/08_deep_research_dispatch.py
```

Reproduce the Offline Default Run II

```
uv run python projects/templates/template_advanced_li
terature_review/scripts/09_export_bibliography.py
uv run python projects/templates/template_advanced_li
terature_review/scripts/05_inject_variables.py
```

Reproduce the Offline Default Run I

`scripts/01b_fixture_phase_replay.py` replays the committed corpus through the phase-provenance contract, writing the per-phase corpora, `phase_metadata.json`, and `cross_phase_analysis.json` without any network access.

Reproduce a Live Retrieval Run I

Refreshing the evidence snapshot is an intentional live operation. The multi-phase search stage reads every phase's queries, engines, and filters from `manuscript/config.yaml` — there is no per-phase command-line surface:

```
uv run python projects/templates/template_advanced_literature_review/scripts/01_multi_phase_search.py
```

then re-run the deterministic downstream stages above and re-inject variables so the manuscript reflects the new evidence. The committed corpus is a dated evidence snapshot; live claims require source-tier provenance and domain review, per the project `AGENTS.md` contracts.

Re-target to Another Topic I

Edit `manuscript/config.yaml` — `project_config.search.term`, `query`, `relevance_keywords`, `subfield_keywords`, `hypothesis_definitions`, and the phase definitions under `project_config.search_phases` (queries, engines, temporal filters, `depends_on`) — then regenerate the seed corpus and re-run. No code changes are required; the manuscript re-targets through token injection.

Live Retrieval I

Enable engines under each phase's `engines` map, supply any optional credentials (Unpaywall email, Semantic Scholar key), and run `scripts/01_multi_phase_search.py`; absent engines degrade to skipped sources. Each phase's engine set is configured independently (the bundled default enables arXiv, OpenAlex, Crossref, and Semantic Scholar per phase, with biomedical archives disabled for the astronomy domain).

Deep Research (Offline Fixture Replay) I

This exemplar also demonstrates the shared `infrastructure.search.deep_research` capability — provider-neutral dispatch to OpenAI and Gemini deep-research agents. Because deep research is a **paid, non-deterministic** service, the template never calls it live in CI. Instead, `src/deep_research/dispatch.py` wires the real infrastructure request/result models (`DeepResearchConfig`, `DeepResearchRequest`, `DeepResearchResult`, `DeepResearchClient`) and ships a deterministic, offline path: `scripts/08_deep_research_dispatch.py` builds the genuine provider-neutral request and then *replays* a recorded report fixture (`src/deep_research/fixtures/recorded_report.json`), normalizing it through the real `DeepResearchResult` model. Replay fails closed if the fixture is missing — it never fabricates a passing run — mirroring the fixture-replay idiom of

`template_sia`. The same source module constructs the exact `DeepResearchRequest` a live `submit` would dispatch, so a fork can enable real providers behind an explicit live-only command:

```
# Offline (default): replays the recorded report, no key required
uv run python projects/templates/template_advanced_literature_review/scripts/08_deep_research_dispatch.py
```

Test Suite I

Every project-owned stage is covered by a no-mocks test suite (real computation and `pytest-httpserver` for network adapters) gated at $\geq 90\%$ coverage on the project-owned `src/multi_phase` / surface (`pyproject.toml` $\rightarrow$ `fail_under = 90`). The suite covers:

Test Suite I

- Multi-phase configuration validation (search, sampling, hypotheses, LLM filters)
- Deterministic filters and phase metadata structure
- Cross-phase overlap (Jaccard) and corpus coverage
- Multi-phase search runner and fixture replay contracts
- Phase-aware manuscript variable extraction (including pending/unmeasured states)
- Deep-research offline replay (provider-neutral, fails closed)
- Publication-extension and fixture-honesty contracts

Test Suite I

Shared analysis, knowledge-graph, reproducibility and visualization methods are imported from the declared `template_literature_meta_analysis` distribution. The Meta project's suite verifies their complete source, while Advanced's consumer contracts verify phase-aware integration. Installed acceptance binds the genuine provider wheel separately from Advanced's project wheel.